



Pune Vidyarthi Griha's
COLLEGE OF ENGINEERING, TECHNOLOGY
AND MANAGEMENT, PUNE-09



Pune Vidyarthi Griha's College of Engineering, Technology and Management

Department of Artificial Intelligence and Data Science

Subject Name & Code:

417532(B): Business Intelligence

Student Name: Shreya Yadav

Roll No.: 22028088

Batch : B3

Date of Submission: 20-04-2026

ASSIGNMENT:- 6

AIM:- Perform the data clustering algorithm using any Clustering algorithm

PROBLEM STATEMENT:- To use the K-Means Clustering algorithm for unsupervised data segmentation. You have a dataset that contains measurements of iris flowers (sepal length, sepal width, petal length, and petal width). Apply the K-Means algorithm to group the flowers into clusters based on feature similarities, so that the natural structure of the data can be identified and different species of iris can be distinguished without using their actual labels.

THOERY:-

Clustering

Clustering is an unsupervised machine learning technique used to group similar data points together based on features or properties. The aim is to ensure that objects within a cluster are more similar to each other than to those in other clusters.

Types of Clustering Algorithms are:

- **Partition-based Clustering** (e.g., K-Means, K-Medoids)
- **Density-based Clustering** (e.g., DBSCAN)
- **Hierarchical Clustering** (e.g., Agglomerative & Divisive)
- **Model-based Clustering** (e.g., Gaussian Mixture Models)

K-Means Clustering Algorithm

K-Means is a centroid-based algorithm and one of the most popular clustering techniques. It partitions the dataset into K distinct, non-overlapping clusters based on distance from the cluster centroid.

How K-Means Clustering Algorithm Works

The steps involved in K-Means Clustering Algorithm include:

1. **Initialization:**
 - Choose the number of clusters K.
 - Randomly initialize K centroids.
2. **Assignment Step:**
 - Assign each data point to the nearest centroid using a distance metric (usually Euclidean distance).
 - This forms K clusters.
3. **Update Step:**
 - Recalculate the centroid (mean) of each cluster based on the new members.
4. **Repeat** steps 2 and 3 until:

- Centroids do not change significantly.
- A maximum number of iterations is reached.
- Convergence criteria is met.

Distance Metric Used

Euclidean Distance:

For points $x = (x_1, x_2, \dots, x_n)$ and $y = (y_1, y_2, \dots, y_n)$,

$$d(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

How to Choose K (No. of Clusters)

- Elbow Method: Plot the Within-Cluster-Sum of Squared Errors (WCSS) against various K values and choose the "elbow point".
- Silhouette Score: Measures how well each point fits in its cluster.

Applications of K-Means Clustering on the Iris Dataset

The Iris dataset contains 150 samples of iris flowers with 4 features: sepal length, sepal width, petal length, and petal width. It includes 3 species: Setosa, Versicolor, and Virginica. This dataset is commonly used for classification, clustering, and pattern recognition tasks.

1. Unsupervised Pattern Discovery:

K-Means helps identify natural groupings of iris flowers based on their physical attributes (sepal & petal length/width), even without knowing the species.

2. Dimensionality Reduction & Visualization:

Clustering helps visualize high-dimensional biological data in 2D using only two features.

3. Data Compression:

In applications like image compression or biological image analysis, flowers can be represented using cluster centroids instead of raw data.

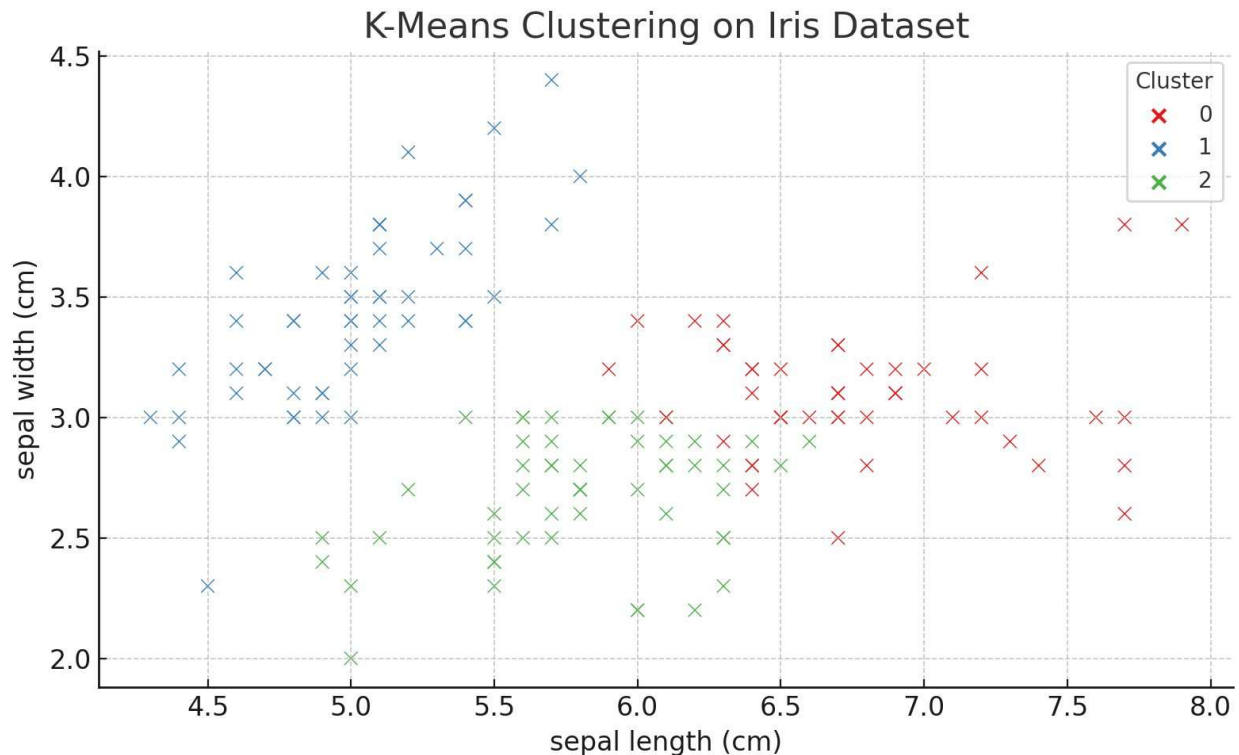
4. Exploratory Data Analysis (EDA):

Before applying classification, K-Means helps in initial understanding of how well data points can be grouped.

Visualization of K-Means clustering

The visualization of K-Means clustering on the Iris dataset :

- The plot shows how the flowers are **grouped into 3 clusters** using the first two features (sepal length and sepal width).
- Each color represents a **different cluster** discovered by the algorithm.
- This helps us visually understand the **natural separation** between different types of flowers.



Advantages of K-Means Clustering

1. **Simple and Fast:** Easy to implement and computationally efficient for large datasets.
2. **Scalable:** Performs well with large datasets due to its linear time complexity.
3. **Unsupervised Learning:** Does not require labeled data, making it ideal for exploratory analysis.
4. **Easily Adaptable:** Works well with various types of numerical data and can be modified for different objectives.

Limitations of K-Means Clustering

1. **Assumes Spherical Clusters:** Works best when clusters are roughly circular in shape and size.
2. **Sensitive to Initial Centroids:** Different initializations may lead to different results.

3. **Needs Predefined K:** You must specify the number of clusters (K) beforehand, which may not be known.
4. **Not Ideal for Outliers:** K-Means is sensitive to outliers which can distort the cluster centers.

CONCLUSION:- K-Means is a powerful and intuitive clustering algorithm used widely for grouping similar data points. On the Iris dataset, it successfully classifies flowers into distinct groups based on their features, showcasing the algorithm's practical utility in unsupervised learning tasks. However, one must carefully choose the number of clusters and preprocess the data properly to achieve optimal results.

CODE:-

```
localhost:8888/notebooks/Untitled97.ipynb
Jupyter Untitled97 Last Checkpoint: 4 minutes ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)
```

```
[1]: # Import necessary Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

[4]: # Load dataset
iris = load_iris()
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)

[5]: # Standardize the data
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df)

[6]: # Find the optimal number of clusters using Elbow Method
wcss = []
K = range(1, 11)
for k in K:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(scaled_data)
    wcss.append(kmeans.inertia_)

[7]: # Plot the Elbow Curve
```

```
Jupyter Untitled97 Last Checkpoint: 4 minutes ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)
```

```
[8]: # From elbow plot, suppose we choose k = 3
kmeans = KMeans(n_clusters=3, random_state=42)
clusters = kmeans.fit_predict(scaled_data)

[9]: # Add the cluster labels to the original dataframe
df['Cluster'] = clusters

[10]: # Show first 5 rows of the result
print(df.head())

  sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1                3.5                1.4                0.2
1                4.9                3.0                1.4                0.2
2                4.7                3.2                1.3                0.2
3                4.6                3.1                1.5                0.2
4                5.0                3.6                1.4                0.2

  Cluster
0        1
1        2
2        2
3        2
4        1

[11]: # Visualize clusters
plt.figure(figsize=(8, 5))
sns.scatterplot(x=df[iris.feature_names[0]],
                y=df[iris.feature_names[1]],
                hue=df['Cluster'], palette='Set1')
plt.title('K-Means Clustering Results')
plt.xlabel(iris.feature_names[0])
```

